

// PROPOSAL · PRELIMINARY RESULTS · MECHANISTIC INTERPRETABILITY

A proposal for interpreting dense networks

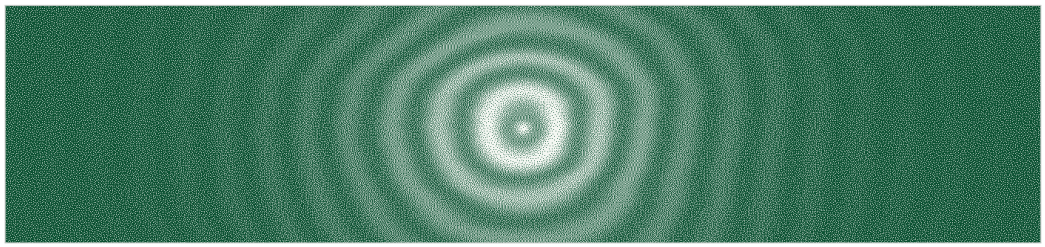
Keira Wiechecki
WAKARU · KAW504@NYU.EDU

Lamb Wakefield
FLOCK CAPITAL · LAMB@FLOCK.HOLDINGS

Jade Zaslavsky
WAKARU · JADEOFNAMELESS@GMAIL

vmfunc
FLOCK CAPITAL · CELESTE@LINUX.COM

October 11, 2026



ORBIT · SEED 1024 · FLOYD–STEINBERG ERROR DIFFUSION

ABSTRACT

Ideally, mechanistic interpretability would provide guarantees about the computational process used by a model: features should be causal, composable, and closed-form. Most existing techniques, such as sparse autoencoders, can perform arbitrarily badly on all three. Arguably the simplest interpretability technique is a linear probe, and the simplest control technique is a steering vector. On current benchmarks both beat features from unsupervised techniques such as SAEs, but they require contrastive data built around a concept someone has already named. In the spirit of MoEfication, we propose a technique that decomposes a representation space into a set of contrastive classifiers. Each class corresponds to a representative vector; inputs are encoded as classifications and reconstructed from the vectors they select. Steering effectively comes for free, outputs are bounded, the representative vectors admit static interpretation in principle, and feature space is structured. Preliminary results on SONAR sentence embeddings and GPT-2 residuals: trained heads are contrast sets; a pure 1900-bit hard code reconstructs at least 33% better than a `TopK` *SAE held to the same total bits, however its coefficients are coded, though an entropy-coded L1 SAE does better still, and on GPT-2 an 800-bit code recovers more of the model's loss than a* k_{SAE}=32 *SAE. Entries are realized by steps in embedding space, and interventions set on heads compose additively. But an input-space step disturbs other heads as much as a random step of its length, past the first layer no head reproduces across seeds, and parameter-space decodes do not describe what an entry does.*

1 Introduction

Ideally we would like mechanistic interpretability to provide guarantees about the computational process used by a model. We would like features to be

1. causal
2. composable
3. closed-form

However most existing techniques, such as sparse autoencoders (SAEs), produce features that can perform arbitrarily badly on these desiderata. As a dictionary grows, features split into narrower ones [5], and a narrow feature can absorb a broader one, which then fails to fire where it applies [10], so the code is not compositional. SAE features can't usually be statically analyzed (though bilinear SAE variants can be). Sparsity trades off against reconstruction fidelity, limiting the extent to which features can be used for steering.

Arguably the simplest interpretability technique is a linear probe, and the simplest control technique is a steering vector. On current benchmarks both beat features from unsupervised techniques such as SAEs, probes at detecting concepts and the difference of means at steering them [40], but they require contrastive data built around a concept someone has already named, labeled or not [6]. In the spirit of MoEfication [43], we propose a technique based on decomposing a representation space into a set of contrastive classifiers. Each class corresponds to a representative vector. Inputs are encoded as a series of classifications, and reconstructed from the representative vectors those classifications select. This has several advantages over SAEs:

1. It effectively gives steering for free: changing a classification is a steering move.
2. Outputs are bounded.
3. The representative vectors admit static interpretation in principle; making that real is currently an open problem (Section 6.8).
4. Feature space is structured.

The steering primitive, measured, is Figure 1; the evaluation so far, including where it cuts against the proposal, is Section 6.

2 Assumptions

This proposal assumes:

- **Steering is inherent to the training objective.** The dictionary vectors must also be high quality steering vectors; interpretability and steering are treated as the same problem.
- **Weights are themselves interpretable** (for certain architectural choices).
- **Clustering rather than decomposition.** Representations are grouped into mutually contrastive categories rather than decomposed into independently firing features.
- **Categorical structure is imposed on features.**

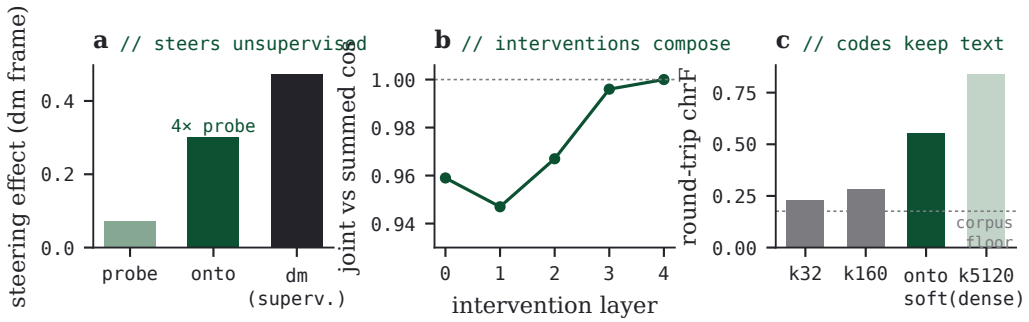


FIGURE 1 · The softmax Ontologizer's steering primitive, measured. (a) Forcing an entry moves decodes along the supervised difference-of-means direction 0.64× as far as supervised steering itself and 4× as far as probe directions (Table 4). (b) Joint two-head interventions point where the summed single interventions point, exactly in the final layer. (c) The sparse SAEs' codes lose most text before any intervention (round-trip chrF 0.23–0.28 against a 0.18 corpus floor) while the soft code keeps it (0.55); the dense k_{SAE}=5120 SAE keeps more but is not a sparse code.

- **Interpretability gets a cheap forward pass.**
- **Concepts compose linearly.** Concepts must be additive to be useful.
- **Dictionary learning is all you need**, in a nonstandard form: the dictionary forms a basis for the representation rather than a target or a discrete codebook. Representations are not themselves quantized; quantization comes from sharpening the classifications over that basis. Trained from scratch, a soft relaxation of quantization need not quantize, since the decoder can learn to invert it [31]; the hard code (Section 6.3) is the response.
- **Interpreting a well trained model is the same problem as interpreting the data and training tasks.**

2.1 Pivotal assumptions

Two of these are pivotal: some version of the linear representation hypothesis [32, 33] is true, and latent space is in some way "natural".

2.2 Concerns

Contrastive classifiers might not be expressive enough This is what the reconstruction benchmarks are for. A soft code pays heavily for its structure, but a pure 1900-bit hard code beats `TopK` SAEs held to the same total bits, though not every sparse linear code (Section 6.3).

Heads might not be identifiable A head can only be named if it is the same head on every run. Past the first layer, no head is: two seeds reach identical error while their heads agree barely above chance (Section 6.6). This is currently the binding constraint on the proposal.

Steering could push it too far off manifold This is an open problem for activation steering. In robot control, a frozen diffusion policy is steered through its input noise rather than its outputs: a reinforcement-learned noise policy, confined to the region the denoiser was trained on, keeps every action within the policy's learned distribution [38]. The analog here is steering through the code, choosing entries and decoding, rather than adding vectors to the embedding. If we can't solve the problem it still gives good data to test steering methods: can we RL a model for high or low scores on a classification and have it converge faster than paired steering or PCA vectors?

Heads could be entangled An intervention set directly on a head composes with others, but a step in input space disturbs other heads as much as a random step of the same length, and the residual cascade doubles that (Section 6.4).

Representations could be too soft/dense/high entropy This is highly correlated across all feature-based interp (SAEs, transcoders [16], J-space). Dividing representations across orthogonal categorical subspaces makes it possible to interpret the conceptual basis vectors even if the representations are dense mixtures. A soft code's argmax is not its code, though (Section 6.1), which is why the hard code exists.

Entries could be too polysemantic Orthogonality and categorical constraints give a theoretical inductive bias to monosemanticity. As long as they have decent explanatory power we can do eigendecomposition; polysemanticity should decrease with parameter count, and there are several directions to increase parameters (width, head count, layers). A short-training scaling sweep found no rise in head coherence with parameter count (working notes); full-length retrains are the missing control.

Interpreting entries could be otherwise intractable This would imply the universality hypothesis is false and is highly correlated with intractability of other interpretability agendas. Submodules could be interpretable even if features aren't.

Training could be intractable A hard code trains, though only with stabilizers whose derivation is in the working notes (Section 6.3).

Doesn't converge in the limit Collapse in one layer would propagate down the stack. Heads do freeze in the softmax models; the working notes trace the cause.

Not orthogonal in the limit The support overlap ω (Section 5.1.3) penalizes heads sharing coordinates but does not guarantee disjoint support at convergence, and the measured answer is that it does not arrive: within a head atoms come apart, but no trained model partitions its coordinates between heads (Section 6.2). A static reading of the dictionary cannot assume disjoint heads; concatenated heads, disjoint by construction, are the route to them.

Might not generalize out of distribution The same concern applies to every feature-based technique. Closed-form interpretation of the dictionary is the mitigation: a statically interpreted basis does not depend on the input distribution, though the classifications over it do.

Inference too expensive Each layer computes all $h \cdot k$ of its classifier logits per sample (Section 5.1), densely: a bilinear classifier costs about twice a dense encoder of the same width, so the code saves no forward-pass compute over an SAE of width $l\ h\ k$. Skipping heads per input, as Switch SAEs route each input to one expert [28], would be needed for that. Training adds the extra decoder passes of deep supervision [25].

Autointerp doesn't work on parameter decodes Parameter-space decodes of entries fail as descriptions outright (Section 6.8). This is direct evidence against the static-interpretability claim.

3 Motivation

3.1 Threat Model

The ontology identification problem. We can't yet reliably look inside a model and draw conclusions about its goals or world model; interpretability has found a model's hidden objective in a planted test, but a semantic search of the training data, with no access to the model, surfaced the same evidence [27]. This is a major limitation on almost all proposed alignment agendas, because behavior alone cannot settle the question: a model can behave differently when it believes it is being trained, and when that belief reaches it through training documents rather than its prompt, the difference can persist with no written reasoning to inspect [19]. Only some models show such a gap [37], but that a good record under test need not predict later behavior is an old argument [3, Ch. 8].

Inner misalignment We don't currently know how to specify a robust goal for a model. Because goal space is large, we should expect goal misgeneralization as the default outcome.

Limits of natural abstractions. Networks trained on the same task can share an algorithm across architectures while differing, even between random seeds, in which features implement it: evidence for weak but not strong universality [11]. In language models, SAE features of a Transformer and a Mamba trained on the same data match nearly as well as those of two Transformer seeds [39]. This is evidence for a weak form of the universality hypothesis, the part of the natural abstraction hypothesis that says many cognitive systems learn similar abstractions [9]; if it holds, it would be a very convenient shortcut for auditing a model.

Natural abstractions could be considered the set of concepts we could use to communicate with aliens. They should consistently emerge through unsupervised learning (in the limit given some constraints). If any neural network is considered a fuzzy generalization of a finite state machine, natural abstractions are the latent states.

Unfortunately, the abstractions a model learns seem highly sensitive to choice of training data, and even to the random seed with the data fixed [11], and are thus not necessarily universal in the limit. [1] Assuming that model capabilities will continue to primarily be driven by access to more data and more modalities, we cannot expect the abstractions learned by a weak model to remain robust in a more powerful model.

Even if we could fully characterise a model's state space in terms of natural abstractions, that still doesn't mean we can reconstruct the latent FSM, as the transition rules can be

[1] The same applies to those that seem most natural to humans.

arbitrarily complex.

3.2 Theory of change

Conditional on the underlying model of ontology being true, the techniques proposed here would be a substantial interpretability advance. The agenda also has a case for counterfactual impact: its theoretical basis consists of nonobvious applications of obscure techniques from niche fields of bioinformatics, and similar research is unlikely to be done soon otherwise.

What a solution would look like. Mechinterp plausibly offers the best path to a robust alignment solution despite criticisms. The case rests on

1. **decomposability** Mechinterp allows alignment to be broken into manageable subproblems.
2. **parallelizability** Decomposable subproblems can be worked on independently.
3. **robustness** Mechinterp offers direct causal explanations of model behavior.

The most straightforward mechinterp alignment solutions are

1. eliciting latent knowledge (ELK) Directly inferring a model's ontology.
2. mechanistic anomaly detection (MAD) Detecting when a model has "unusual" thoughts.

Both of these can be considered ontology identification subproblems. MAD requires mapping between the model's activations and the model's ontology. ELK requires mapping between the model's ontology and human ontology. These are hard but not intractable problems. Recently, developmental interpretability (devinterp) techniques have divided training into "developmental stages", each marked by a change in the network's internal structure and behavior, such as learning bigram statistics or forming induction heads [20]. This is suggestive evidence, from two small transformers, that learning consists primarily of the network grokking [30] discrete concepts. While these results are promising for MAD, locating these emergent capabilities in the model's activations remains an open problem.

Interpretability glosses broadly as "let humans understand AI". Getting AI to understand humans better scales badly: making the model smarter leads to subtler and subtler issues with larger and larger effects. Between the two sits scalable oversight, in which models help humans supervise models: people working with an unreliable dialog assistant already outperform both the assistant and themselves alone [4], and interpretability is one of the channels such oversight can draw on. And what we're doing is not just letting humans understand AI. Because an LLM reflects humanity from its training data, high quality conceptual interpretability provides machine-comprehensible mathematical representations of every concept that humanity cares about. We work on interp for the usual corrigibility reasons, and also because outer alignment needs a stable place to attach values: a goal defined over one of a model's concepts must keep pointing at that concept as the model's representations are revised [3, Ch. 12]. The preliminary results measure a small version of this problem: past the first layer, no head survives a change of seed (Section 6.6). This attacks both.

Macro-scale strategy. Broadly, the goals are to

1. formalize the notion of a model's latent concept space
2. distill a model's latent concept space
3. enforce decomposability of latent concept space
4. enforce robustness of existing concepts as the dimensionality of the latent concept space increases

A prerequisite for all of these is the ability to express a metaontology which can identify latent concepts in activation space.

4 Background

Mechanistic interpretability currently can do supervised steering well or unsupervised steering poorly. The Google DeepMind mechanistic interpretability team has pivoted from ambitious reverse-engineering to pragmatic interpretability judged on proxy tasks, and has deprioritized SAE research: SAE probes did not generalize better out of distribution, and SAEs did not beat a steering vector built from one contrastive pair of prompts at suppressing a model's awareness of being evaluated [29]. On current benchmarks the supervised difference of means steers better than SAE latents, though worse than prompting [40]. But a supervised method finds only the concepts an experimenter already has labels for, and a direction that changes outputs as intended need not be one the model computes with: a subspace intervention optimized for its effect can instead activate a dormant pathway [26]. The dominant unsupervised paradigm is the sparse autoencoder (SAE). A model's hidden state is approximated as a finite state machine consisting of discrete features. The first implementations used a `ReLU` activation and sparsity loss [14, 5], though more recent implementations have used a `TopK` activation function without a sparsity loss [18].

4.1 Common problems with SAEs

The motivation is to decompile a model to a finite state machine (FSM). The limitation is that such an FSM is only as interpretable as an enumeration of all possible states. Even a mixture of two structureless processes, one constant and one a fair coin, needs infinitely many states to track which component is generating, so the structure lives in how the components overlap [13].

Features are not composable. A persistent problem with SAEs is feature absorption [10]: when one feature implies another, sparsity is better served by folding the general feature's direction into the specific feature's latent and switching the general latent off there, so a latent that looks monosemantic silently fails to fire on some inputs that have its feature (a "starts with S" latent that is silent on "short"). A related failure, composition, learns co-occurring concepts as one latent. Matryoshka SAEs [7] reduce both by training nested prefixes of a single dictionary to reconstruct the input on their own, so early latents learn general features and later ones specific features. Tree SAEs [8] make the hierarchy explicit by gating each latent on an assigned parent.

Features are not cleanly causal. Sparsity trades off against reconstruction fidelity [18]. Steering with an SAE feature is rarely clean: adding its decoder direction moves the logits differently in different contexts and disturbs unrelated downstream features. How much varies by feature and is partly predictable before intervening, from the crowding of its decoder direction and its direct effect on the logits, though which predictor works differs between models and dictionaries [15].

Features are not closed-form. A major concern is how well an interpretability technique generalizes out-of-distribution. Ideally we would like to infer behavior by looking solely at model weights. Bilinear MLPs make this partly possible: the interaction matrix of any output direction can be eigendecomposed without data, although in deep models the output directions to decompose still come from data-dependent SAEs [35].

Forward passes are expensive. Efficiency is also a problem. Although the sparsity constraint means that only a handful of features are active for a given input, the entire feature space needs to be computed for each sample [18, 28]. Mixture-of-experts (MoE) SAE variants reduce compute costs by dividing feature space between smaller submodules. Examples include Switch SAEs, which route each input to one expert [28], and Scale SAEs, which activate several to curb features duplicated across experts [41].

Less explored is the effect of the structure imposed by the MoEfied architecture on the interpretability of feature space. A common approach to scalably interpreting features is to have an LLM explain each feature from its activating examples and score the explanation by how well an LLM judge, given only the explanation, separates activating from non-activating contexts [34]; in the results here one model both writes and judges the explanations. With an unstructured feature space, it quickly becomes intractable to interpret all permutations of features this way. It is however possible to instead apply this technique to expert routings. The ReLU feed-forward layers of a trained transformer can themselves be split into experts of co-activating neurons, keeping over 95% of performance with 10–30% of the parameters [43]. Language models appear to represent a categorical concept as a simplex whose vertices are the vectors of its values, with hierarchically related concepts in orthogonal subspaces, so that a hierarchy is a direct sum of simplices; differences between value vectors are the steering vectors of binary contrasts [33]. We conjecture that MoEfication into classifiers can reveal such latent contrastive steering vectors, since an entry swap writes exactly such a difference. Penalizing the cosine between each SAE decoder direction and its nearest neighbor cuts measured feature absorption by about two thirds relative to BatchTopK SAEs, at slightly lower explained variance and unchanged autointerp scores [23]; that penalty is unstructured, where the Ontologizer asks for orthogonality between heads and contrast within them.

Existing attempts to route activations to expert SAEs focus on routing similar inputs to the same expert [28], and score an expert as specialized when its features resemble each other and not other experts' [41]. Contrastive learning wants the opposite: training on things at opposite ends of a shared axis. We don't just want to find similar features, we want to find contrastive vectors in feature space that correspond to what a model considers a natural category.

5 Methods

5.1 Architecture

We propose a method that encodes samples as classifications mapped to contrastive vectors. This directly constrains a model's representations, and classifications can natively be used for steering.

The model (Figure 2) consists of l `DictEnc` layers writing into a shared residual accumulator, decoded by a single linear map. Each layer (Figure 3) is a classifier followed by a dictionary. The classifier splits its input into h independent *assignments*, each a distribution over one head's k *entries*, and the dictionary maps each entry to an *atom*. Each layer writes the assignment-weighted sum of its atoms to the accumulator; after each layer the accumulator is decoded and the residual passed on, so layer ℓ receives $\mathbf{x} - \mathrm{dec}(\mathbf{r}_\ell)$, where $\mathbf{r}_\ell$ holds the ℓ layers before it. Deep supervision [25] constrains every prefix of the stack to reconstruct the input.

Because entries within a head are mutually exclusive rather than independent, each head is a contrastive set: a k-way classifier whose entry swaps are steering moves. A between-head penalty is meant to push each head onto its own coordinates of the dictionary space (Figure 4); measured on the trained models, it did not (Section 5.1.3).

Classifier. Each `DictEnc`'s classifier is a multihead bilinear map (`BilinearBlock`) with weights $[\mathsf{W}, \mathsf{V}] \in \mathbb{R}^{2\times h\times k\times d}$ ($d+1$ with the constant coordinate of Equation 9). It reads the shape $\mathbf{u}_\ell$ of the layer's input (the residual left by the layers before it, or $\mathbf{x}$ at layer 0), written $\mathbf{x}$ below for a generic input, and returns logits $K \in \mathbb{R}^{h\times k}$,

$$\texttt{BilinearBlock}([\mathsf{W}, \mathsf{V}], \mathbf{x}) := \left(\sum_{q=1}^{d} W_{\cdot\cdot q}\, x_q\right) \odot \left(\sum_{q=1}^{d} V_{\cdot\cdot q}\, x_q\right). \tag{1}$$

Each logit is a quadratic form in $\mathbf{x}$, so a classifier admits closed-form eigendecomposition in the manner of bilinear MLPs [35].

Assignments and selection. Soft assignments are $P = \mathrm{softmax}_k(K/t)$, where the temperature t is annealed to sharpen them. A soft code's argmax is not, however, what the model decodes: Section 6.1 finds the softmax model's discrete reading far from its embedding, as a soft relaxation of quantization trained from scratch need not quantize when the decoder can learn to invert it [31]. The *straight-through* selection rule makes the discrete code the model: the forward pass selects the one-hot argmax, and the backward pass uses $\mathrm{softmax}(K/t)$ as its surrogate gradient, the straight-through estimator [2] in its categorical form [21]. A hard code transmits exactly $l\, h \log_2 k$ bits, and its only continuous numbers are the layers' gains g_ℓ.

From the assignments P_n of each sample n we collect per-sample entropy, in bits,

$$\mathrm{entropy}_k(P)_i := -\sum_{j=1}^{k} p_{ij} \log_2 p_{ij}, \qquad H := \mathbb{E}_n\, \mathbb{E}_i \left[\mathrm{entropy}_k(P_n)_i\right], \tag{2}$$

between-sample similarity, the mean absolute cosine of the per-head features over pairs of samples, with $\cos\angle(\mathbf{a}, \mathbf{b}) := \langle \mathbf{a}, \mathbf{b}\rangle / (\|\mathbf{a}\|\|\mathbf{b}\|)$ the cosine of the angle between two vectors,

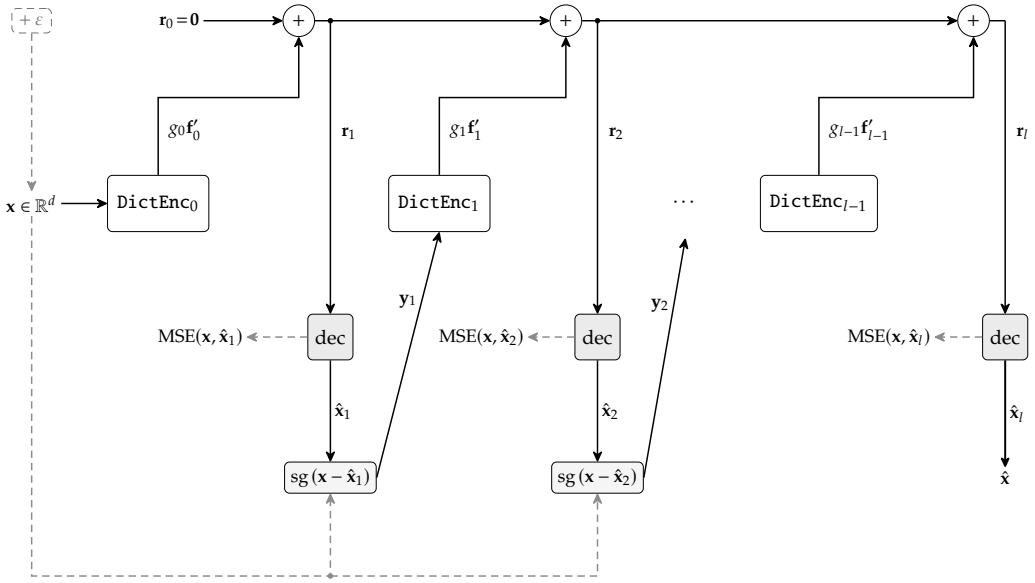


FIGURE 2 · **Ontologizer architecture.** An `Ontologizer` consists of l stacked `DictEnc` layers. Each returns a pooled feature vector $\mathbf{f}'_\ell \in \mathbb{R}^e$, which is written, scaled by its input's gain g_ℓ, to a residual accumulator $\mathbf{r}$. Layers are indexed $\ell = 0, \ldots, l-1$ and accumulator states by the number of layers applied: $\mathbf{r}_0 := \mathbf{0}$ and $\mathbf{r}_{\ell+1} = \mathbf{r}_\ell + g_\ell\, \mathbf{f}'_\ell$, so layer ℓ reads state ℓ and writes state $\ell + 1$; a stop-gradient variant (sg on $\mathbf{r}_\ell$, available but off in every model reported here) would prevent gradient communication between layers. dec is a linear decoder with shape $\mathbb{R}^{d\times e}$, shared by all layers. Deep supervision makes the code a successive refinement, coarse-to-fine in distortion as in residual quantizers [42] and nested-prefix SAEs [7]: the component encoded by previous layers is inaccessible to later layers. After ℓ layers, $\hat{\mathbf{x}}_\ell = \mathrm{dec}(\mathbf{r}_\ell)$, and the component left unexplained is $\mathbf{y}_\ell = \mathbf{x} - \hat{\mathbf{x}}_\ell$, so $\mathbf{y}_0 = \mathbf{x}$. Layer ℓ classifies the shape of its input, $\mathbf{u}_\ell = \mathbf{y}_\ell / \mathrm{L2}(\mathbf{y}_\ell) \oplus [1]$, and scales what it writes by the gain $g_\ell = \mathrm{L2}(\mathbf{y}_\ell)$. Normalizing prevents exploding gradients from repeated application of bilinear layers, each of which scales its input by $\|\cdot\|^2$; the constant coordinate keeps the sign of $\mathbf{y}_\ell$, which the bilinear form would otherwise lose. $\mathrm{MSE}(\mathbf{x}, \hat{\mathbf{x}}_\ell)$ is computed separately for each prefix $\ell = 1, \ldots, l$. The input-noise hook drawn on $\mathbf{x}$ is disabled in every model reported here.

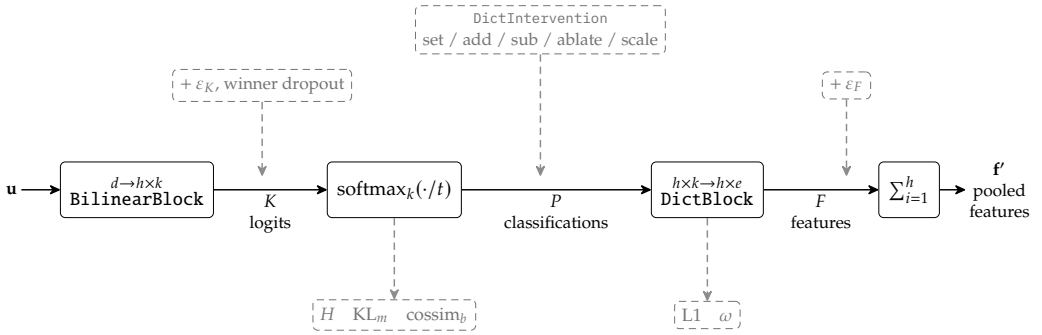


FIGURE 3 · **DictEnc architecture.** Dashed lines above indicate perturbations to the data, including noising and causal interventions. Dashed lines below indicate statistics used as tuneable auxiliary loss functions. Each DictEnc consists of a multihead bilinear classifier (§5.1.1) and a multihead dictionary (§5.1.3). Noise ε_K and ε_F and winner dropout regularize training (§5.1.5). Assignments P are softmax$_k(K/t)$, or its straight-through argmax (§5.1.2); interventions act on P. From P we collect per-sample entropy H, between-sample similarity cossim$_b$ and batch-mean entropy KL$_m$ (Equations 2–4). A DictBlock maps $P \in \mathbb{R}^{h \times k}$ to features $F \in \mathbb{R}^{h \times e}$ (Equation 5); from P and the atoms we collect the support overlap ω (Equation 7), and from F the sparsity penalty L1. F is summed over heads to give pooled features $\mathbf{f}' \in \mathbb{R}^e$, which are added to the accumulator $\mathbf{r}$.

evaluated through each head's dictionary Gram $G_i = |D_i||D_i|^\top$,

$$\text{cossim}_b := \mathbb{E}_{n,n'} \left| \cos \angle(\mathbf{f}_n, \mathbf{f}_{n'}) \right|, \qquad \langle \mathbf{f}_n, \mathbf{f}_{n'} \rangle = \sum_{i=1}^{h} \mathbf{p}_{ni}^\top G_i \, \mathbf{p}_{n'i}, \tag{3}$$

and the divergence of the batch-mean assignment from uniform,

$$\text{KL}_m := \mathbb{E}_i \left[\log_2 k - \text{entropy}_k(\mathbb{E}_n[P_n])_i \right]. \tag{4}$$

KL$_m$ anchors each head's mean assignment to uniform, which keeps every entry in use.

Dictionary. A DictBlock with weights $\mathsf{D} \in \mathbb{R}^{h \times k \times e}$ maps assignments to per-head features $F \in \mathbb{R}^{h \times e}$, whose row $\mathbf{f}_i$ is head i's output,

$$\mathbf{f}_i := \sum_{j=1}^{k} p_{ij} \, \mathbf{a}_{ij}, \qquad \mathbf{a}_{ij} := |\mathsf{D}_{ij\cdot}|. \tag{5}$$

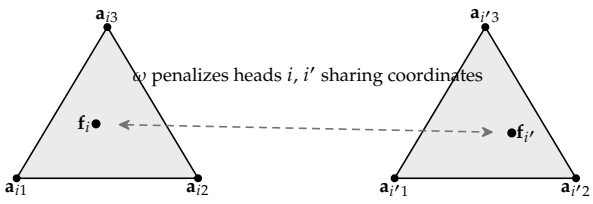
Atoms are non-negative and assignments lie on the simplex, so features are strictly additive and each head's output lies in the convex hull of its atoms (Figure 4).

Disjoint support For non-negative vectors the inner product is a sum of non-negative terms, so it vanishes exactly when the supports are disjoint,

$$\langle \mathbf{u}, \mathbf{v} \rangle = 0 \iff \text{supp}(\mathbf{u}) \cap \text{supp}(\mathbf{v}) = \emptyset. \tag{6}$$

Between heads, then, orthogonality is not a rotation constraint but a support partition. The support overlap compares heads' usage-weighted coordinate profiles,

$$\boldsymbol{\pi}_i := \sum_{j=1}^{k} \bar{p}_{ij} \, |\mathbf{a}_{ij}|, \qquad \omega := \mathbb{E}_{i \neq i'} \frac{\langle \boldsymbol{\pi}_i, \boldsymbol{\pi}_{i'} \rangle}{\|\boldsymbol{\pi}_i\| \|\boldsymbol{\pi}_{i'}\|}, \qquad \bar{\mathbf{p}}_i := \mathbb{E}_n \left[\mathbf{p}_{ni} \right], \tag{7}$$



$$\mathbf{f}' = \textstyle\sum_{i=1}^{h} \mathbf{f}_i, \qquad \mathbf{f}_i = \textstyle\sum_{j=1}^{k} p_{ij}\, \mathbf{a}_{ij}, \qquad \textstyle\sum_{j=1}^{k} p_{ij} = 1$$

FIGURE 4 · `DictBlock` **head geometry** for h=2; k=3. The output space of a head i is a simplex with its atoms $\mathbf{a}_{ij}$, the rows of $|D_i|$, as vertices. The assignment P is a soft choice among mutually contrastive categories rather than a set of independently firing features, and because $p_{ij} \in [0, 1]$ the output lies within the convex hull of the head's atoms. Because $|\mathsf{D}| \geq 0$ and $P \geq 0$, $F \geq 0$, so $\cos \angle(\mathbf{f}_i, \mathbf{f}_{i'}) \in [0, 1]$, and driving the support overlap ω to zero asks the heads to partition the coordinates of $\mathbb{R}^e$ between them (§5.1.3).

and is zero exactly when no two heads share a coordinate. Because a softmax gives every entry positive weight, a single sample on which two heads are orthogonal forces every pair of their atoms to be disjoint: the partition is a property of D, holding for all inputs, which is the premise a static reading of the dictionary needs. Under `ConcatDictBlock` heads write disjoint slices by construction and ω is identically zero.

Two orthogonalities are in play. Disjoint support assigns each head a block of coordinates; *within* a head the k atoms are contrastive alternatives spread apart inside that block, which Section 6.2 measures. Neither is guaranteed at convergence: ω is a penalty, and the geometry makes a support partition the cheap solution rather than forcing one. Measured, the within-head orthogonality holds and the between-head partition does not form (Section 6.2).

Decoder. Each layer's pooled output $\mathbf{f}'_\ell = \sum_{i=1}^{h} \tilde{\mathbf{f}}_i$ is scaled by its input's gain g_ℓ and added to the accumulator, whose state after ℓ layers a single unbiased linear map shared by all layers decodes,

$$\mathbf{r}_\ell = \sum_{\ell'=0}^{\ell-1} g_{\ell'}\, \mathbf{f}'_{\ell'}, \qquad \hat{\mathbf{x}}_\ell = W_{\text{dec}} \mathbf{r}_\ell, \qquad W_{\text{dec}} \in \mathbb{R}^{d \times e}. \tag{8}$$

The residual $\mathbf{y}_\ell$, layer ℓ's input, is stop-gradiented, so that lower layers cannot write a communication code into it instead of reducing it, and split into a gain and a shape $\mathbf{u}_\ell$, its unit direction with a constant coordinate appended, which is what layer ℓ classifies:

$$\mathbf{y}_\ell = \text{sg}[\mathbf{x} - \hat{\mathbf{x}}_\ell], \qquad \mathbf{u}_\ell = \frac{\mathbf{y}_\ell}{\text{L2}(\mathbf{y}_\ell) + \epsilon} \oplus [1], \qquad g_\ell := \text{sg}[\text{L2}(\mathbf{y}_\ell)]. \tag{9}$$

The empty sum makes $\mathbf{r}_0 = \mathbf{0}$, so $\mathbf{y}_0 = \mathbf{x}$: layer 0 classifies the input itself. Under `resid_gain` each layer classifies the direction of what it corrects, and the gain carries the magnitude it cannot see; without it $g_\ell = 1$.

Direct atoms and fibers. Two variants change what an entry writes. With *direct atoms* the dictionary is signed and lives in the output space itself, with no decoder, so every atom is literally a steering direction. With *fibers* each entry carries a rank-r local basis, and each head reads r coordinates from its input, so a hard code becomes a discrete base point plus

a continuous correction within that entry's fiber; training the fibers after freezing the base keeps the discrete code exactly what it was.

Noise and dropout. Classifier logits take Gaussian noise ε_K, relative to the logit scale under a hard forward; features take noise ε_F proportional to each feature's batch standard deviation, stop-gradiented so the model cannot evade it by shrinking variance. Winner dropout sets a head's argmax logit to $-\infty$ with probability p_{drop}.

5.2 Training

Embedding model. We embed texts using SONAR [17], a multimodal and language-agnostic sentence embedding model: a translation-trained encoder, mean-pooled into one 1024-dimensional vector, and a decoder that generates text from that vector alone, in a target language chosen at decoding time. Translations embed close together, so a reconstructed or steered embedding can be decoded in one fixed language and re-encoded, which is how text fidelity and text-cycle steering are scored. Large Concept Models [24] generate one sentence embedding per step in SONAR space, so an edit to a SONAR embedding is an edit to an LCM's context or output; the same work finds SONAR decoding fragile to small perturbations. GPT-2 small's residual stream [36] is a second substrate, where sparse dictionary learning is known to work and the reconstruction can be spliced back into the model.

Loss function. Reconstruction is trained with a whitened MSE, $\mathbf{w}$ being the per-dimension inverse variance normalized to mean one,

$$\mathrm{MSE}(X, \hat{X}) := \frac{1}{bd} \sum_{n=1}^{b} \sum_{q=1}^{d} w_q \left(\hat{x}_{nq} - x_{nq}\right)^2, \tag{10}$$

under deep supervision [25]: it is computed at every prefix $\hat{X}_\ell$, $\ell = 1, \ldots, l$, and averaged, so each layer must explain what earlier layers left unexplained. The auxiliary statistics are weighted and summed over layers,

$$L := \frac{1}{l} \sum_{\ell=1}^{l} \mathrm{MSE}(X, \hat{X}_\ell) + \sum_{\ell=0}^{l-1} \mathbf{s} \cdot \boldsymbol{\psi}_\ell, \qquad \boldsymbol{\psi}_\ell := \left[\, \mathrm{L1} \;\; \mathrm{cossim}_b \;\; \mathrm{KL}_m \;\; \omega \,\right]_\ell^\top, \tag{11}$$

with L1 the batch mean of each sample's feature norm. In practice cossim_b also minimizes H, which therefore carries no weight. Every model reported here trained before ω existed, with its predecessor in that slot: the mean cosine between heads' features within a sample, which for non-negative features stands in for the same disjointness.

5.3 Evaluation

Evaluation runs against `TopK` SAEs [18] trained on the same cached embeddings under the same whitened objective, scored by whitened fraction of variance unexplained (FVU_w) on a held-out tail. A structural-ablation ladder of SAE variants adds one Ontologizer commitment at a time; two of its rungs, which add per-head competition over a shared decoder, are one-layer Ontologizers with linear classifiers (`g160top1` with hard selection, `g160softmax` with softmax) and are reported as simplified variants rather than baselines.

The headline instruments are reconstruction against code capacity and a Gaussian rate–distortion reference [12]; the semantic coherence of a head's decoded atoms against size-matched nulls; steering scored in embedding space (whether a step realizes the target entry, and what share of other heads it disturbs, against a random step of the same length); additivity of paired interventions; cross-seed head matching by partition and by contribution; and blinded auto-interpretability. AxBench [40] found that prompting, finetuning and one simple supervised direction, the difference of means, beat SAE latents at steering, while a linear probe's direction did not; the steering comparison here, which includes both, is built to face that result.

6 Preliminary results

The evaluation is in progress, and its results are mixed. This section gives the headline of each line of work; the full methodology, the eliminations behind each result and the supplementary measurements are in the working notes[2].

Three models recur. The softmax Ontologizer `resid_nc` has l=5 layers of h=32 heads with k=32 entries, a soft code. The straight-through Ontologizer `ste_h76_init01` has 5×76 heads at k=32, a hard code of 1900 bits. The SAEs are `TopK` SAEs on the same cache and objective. The cache holds SONAR embeddings of an 86-language mC4 sample, 93% of them encoded under SONAR's English source tag rather than their own, which moves an embedding by about twice the hard code's reconstruction error but changes how much of it language explains only from 7.4% to 6.7%. Scores are FVU_w on the final 32,768 cache rows, which the SAEs and the straight-through arms never train on. The softmax Ontologizers predate that holdout, but their scores move by at most 0.3% on freshly encoded rows that no model saw.

6.1 The price of a soft code

Contrast-set structure is expensive in a soft code. The plain-SAE frontier dominates the softmax Ontologizer's deviation code (each head's v entries furthest from the mean assignment kept, the rest pinned to it) at every capacity measured (Table 1): at $\mathrm{FVU}_w \approx 0.49$ the SAE transmits 32 coefficients where the Ontologizer needs 320, and at matched budgets an unstructured code reconstructs about 50× better. The soft model's own discrete reading is not a code at all: its argmax lands 77× above the least distortion an 800-bit code could reach if the embeddings were Gaussian with their covariance. Real data is no harder to compress than a Gaussian of the same covariance [12], so that is a lower bound on its distance from the least distortion any 800-bit code could reach on this data (a limit approached by coding long blocks of samples jointly), and the entries an intervention forces are not what carries the soft model's embedding. That is the motivation for the hard code.

6.2 Heads are contrast sets

Trained heads carry the signature the steering goal predicts. Every entry has an exact decode direction in output space, and within a soft head those directions repel: in 96% of the softmax Ontologizer's heads they are more dissimilar than size-matched random sets

[2] K. Wiechecki et al., *Dictionary learning as unsupervised steering-vector generation: Ontologizers vs. sparse autoencoders on SONAR sentence embeddings*, working notes, in preparation; distributed with this proposal.

point	coeffs	index bits	FVU_w
onto v=0 (mean-assignment origin)	0	0	1.001
onto hard (argmax)	0	800	17.34
Gaussian reference, 800 bits	0	800	*0.226*
SAE k_{SAE}=32 tier	32	276–313	0.494–0.509
onto dev v=1	160	800	0.677
SAE k_{SAE}=160	160	1206	0.258
onto dev v=4	640	2421	0.289
onto dev v=8	1280	3732	0.094
SAE k_{SAE}=5120 (L0≈1520)	1520	6423	**0.00011**
1-layer soft variant `g160softmax`	5120	0	0.00436
onto soft (v=32)	5120	0	0.00538

TABLE 1 · Reconstruction against code capacity for the softmax Ontologizer `resid_nc`. Ontologizer points truncate each head to its v largest deviations from the mean assignment; SAE points use measured L0. Index bits name which coefficients are sent, as a set: $l\,h\log_2\binom{k}{v}$ for the Ontologizer and $\log_2\binom{m}{\mathrm{L0}}$ for an SAE of m latents. The Gaussian reference is reverse water-filling on the evaluation tail's whitened covariance [12, Thm. 10.3.3]: the least distortion at that rate for a Gaussian source of that covariance, which real data can beat.

heads or grouping	count	mean $\zeta_{\cos}$	frac ζ>2	frac ζ<−2
`g160top1` (trained, hard)	160	+161.3	1.00	0.00
k_{SAE}=32 SAE, discovered groups	359	+1.7	0.36	0.02
Ontologizer heads (trained, soft)	160	−17.6	0.04	**0.96**
`g160softmax` (trained, soft)	160	−14.7	0.00	**1.00**

TABLE 2 · Within-head decode-direction coherence $\zeta_{\cos}$, standardized against size-matched same-pool nulls (n_{null}=500). Soft heads are anti-coherent: their atoms are pushed apart.

of atoms from the same pool (Table 2). A head holds well-separated alternatives along one dimension, not a semantic cluster, and the repulsion strengthens under the training objective's whitened inner product. The hard code agrees from the weights: a head's decoded atoms are orthogonal to slightly repelling (mean cosine 0.023 at layer 0, −0.015 to −0.020 below). Hard per-head selection in a single linear layer gives the opposite signature, heads that tile one region.

The other orthogonality the architecture was designed for did not emerge. No trained model partitions its dictionary space between heads: the support overlap ω (Equation 7) sits at or above a null that shuffles each head's coordinates, in every model measured, and each head's usage covers 52–98% of the coordinates where a partition into 32 heads needs about 3%. This holds for the part of each atom the decoder sees as well as for its null-space part. Heads are contrast sets that share their coordinates.

6.3 A discrete code beats TopK SAEs, not every sparse code

Under straight-through selection the forward pass is the argmax, and the discrete code is the model. At 1900 index bits the hard code reaches FVU_w 0.154. Its only continuous numbers are four per-layer gains, and with those pinned to their means it is a pure 1900-bit code at 0.158. A code that also sends coefficients reaches that total only at some precision for them, and no coding of an SAE's coefficients beats the least-squares fit on its own

code	coeffs	index bits	FVU_w	at 1900 bits	every row
straight-through Ontologizer, 5×76	**4 gains**	**1900**	**0.154**	**0.155**	—
gains pinned to their means	0	1900	0.158	—	0.158
SAE, `TopK`, k_{SAE}=160	160	1206	0.258 (0.236)	0.259	0.258
SAE, ReLU+L1 3×10^{-5}	519	2418	0.219	0.242	0.250
SAE, ReLU+L1 10^{-5}	1438	4380	0.048	0.175	0.200
SAE, ReLU+L1 3×10^{-6}	1637	4623	0.005	**0.129**	**0.152**
1-layer hard variant `g160top1`	160	800	0.274 (0.273)	0.274	0.274

TABLE 3 · The hard code against converged SAEs on the held-out tail. In parentheses, the least-squares fit on each code's own support, the least error any coding of its coefficients can reach. "At 1900 bits" is each code's error at a total of 1900 bits per sample averaged over rows, with everything it sends quantized and each slot's symbols entropy-coded; "every row" holds each row to 1900 bits, the pinned hard code at fixed length. `g160top1` shares the hard per-head selection but has one layer of linear classifiers and keeps a magnitude on each winner; its index bits name each head's winner.

support. A `TopK` SAE that names its 160 active latents in 1206 bits totals 1900 bits at 4.3 bits per coefficient, where its error is at least 0.236, so at matched total rate the pure code is at least 33% below it (Table 3). An SAE whose sparsity is not fixed does better: with every code's coefficients quantized and each slot's symbols entropy-coded, an L1 SAE with a weak penalty reaches 0.129 at 1900 bits averaged over rows, because its latents fire unevenly and are cheap to name and its rows vary in length, while the hard code uses its entries almost uniformly in a code of fixed length. Held to 1900 bits on every row the L1 SAE reaches 0.152, level with the hard code, and charging every support its fixed-length cost, the hard code leads (0.158 against 0.180). The stack sits 3.1× above the Gaussian reference for its rate, so at least that far above the data's distortion–rate function at 1900 bits, a long-block limit that a code describing each sample alone need not approach [12].

6.4 Depth helps reconstruction and costs addressability

Five layers of 76 heads and one layer of 380 have the same bits and exactly the same parameter count, and every head in both uses its entries almost uniformly. The stack's error is 1.79× lower (0.1535 against 0.2750), and the gain comes from pairing each stage's input with the sample's own earlier error: a flat layer decodes like an additive quantizer [1], but its heads choose in parallel rather than each against what the others leave, so no head sees another's error (harmless in a product quantizer [22], whose heads own disjoint coordinates, but not when atoms overlap), while the stack is a residual quantizer [42]. The stack also trains each of its prefixes to reconstruct, which a single layer cannot, and that difference is not yet separated from the pairing.

The same cascade entangles the heads. A step in embedding space along a target entry's decode direction, a quarter of the input's norm long, realizes the target on 99.6% of rows in the flat model while disturbing 33% of the other heads; in the stack it realizes 37% while disturbing 66%. In both, every direction disturbs as many other heads as a random step of the same length: the hard code's heads sit almost on their decision boundaries (median winner–runner-up margin 1.5×10^{-4}), and the cascade adds downstream layers reclassifying a changed residual. The one Ontologizer whose decode direction avoids the other heads is the simplest, `g160top1`: it realizes 58% of targets while disturbing 2.6% of

arm	eff (native)	eff (dm frame)	hit rate	collateral
difference of means	0.474	0.474	0.062	0.752
entry forcing	0.399	0.302	0.000	0.776
linear probe	0.294	0.072	0.000	0.701
random direction	–	–	–	0.705

TABLE 4 · Language steering under matched intervention budgets on the softmax Ontologizer `resid_nc_hm`; medians over 24 language–magnitude conditions per arm. Effect is displacement along the arm's own direction (native) and along the shared difference-of-means direction (dm frame); hit rate is the fraction of steered decodes reaching the median projection of genuine target-language rows; collateral is content change elsewhere, with the random direction as its floor.

other heads, against 23% for a random step.

6.5 Steering

Steering works when it is measured where it acts. In the soft model a decode-direction step of a quarter of the input's norm realizes the target entry on 82% of rows, against 2% for a random step of the same length; the text round trip had hidden this, since an unsteered decode–re-encode cycle preserves a head's choice on only 11–13% of rows. Interventions set directly on a head compose: the decode change of a joint two-head intervention points where the sum of the single interventions points (median cosine at least 0.95 in every layer, exact in the last, where nothing downstream can reclassify). Forcing one first-layer entry of `resid_nc` reliably steers decodes into Burmese while preserving their content, a causally verified existence proof on one seed.

Against supervised steering on language, under matched intervention budgets (Table 4), forcing an entry moves decodes along the supervised difference-of-means direction 0.64× as far as supervised steering itself and 4× as far as linear-probe directions. Every arm's collateral, supervised steering included, sits at the random-direction floor.

6.6 The error reproduces; the heads do not

Two hard-code runs differing only in seed reach identical held-out error, 0.1535 and 0.1535. Nothing that identifies an individual head agrees: matched head partitions score a mean NMI of 0.012 against a 0.005 null, and head contributions a centered cosine of 0.007 against 0.002, while a run against its own earlier checkpoint scores 0.38 on partitions. Nor do the two seeds compute one reconstruction: their whitened residuals correlate at 0.30, where the two seeds of a k_{SAE}=32 SAE reach 0.83. Within one run each head keeps its atoms (cosine 0.98–0.99 to its own checkpoint 19,500 steps earlier), so what fails is replication across seeds, not stability in training. GPT-2 replicates the pattern: only the first layer reproduces, at under a fifth of a same-run control. The softmax Ontologizer fares no better, matching at most 0.1% of entries across seeds where the k_{SAE}=32 SAE pair matches 3–31%. The first layer is the only one whose input does not depend on upstream choices, and one head in it does recur: the strongest head in every run, found at different indices in different seeds, encodes topic and genre (its entries decode to crime news, geopolitics, encyclopedic description, manufacturing, marketing, legislation and entertainment, and contrastive descriptions of them are detected at F1 0.52 against a 0.27 null). Language, the corpus's one labelled factor, is carried as neighbourhoods across heads and never by one,

model	code per token	FVU	loss recovered
hard code, direct atoms	800 bits + 5 gains	0.195	**97.6%**
hard code, private heads, k=64	960 bits + 5 gains	**0.164**	**97.9%**
`TopK` SAE, k_{SAE}=32	32 coeffs + ~276 bits	0.189	96.2%
`TopK` SAE, k_{SAE}=128	128 coeffs + ~859 bits	0.116	98.6%

TABLE 5 · GPT-2 small, residual stream entering block 8, held-out documents. Loss recovered is $(\text{CE}_{\text{ablate}} - \text{CE}_{\text{recon}})/(\text{CE}_{\text{ablate}} - \text{CE}_{\text{clean}})$ with the residual mean-ablated as the floor; FVU is unwhitened here. Private heads write disjoint 48-dimensional blocks through a shared decoder. One seed per Ontologizer.

as a language-agnostic embedding should give.

6.7 Hard codes on GPT-2

On GPT-2 small, with signed atoms in activation space and no decoder, an 800-bit hard code recovers 97.6% of the model's loss when its reconstruction is spliced back in, against 96.2% for a k_{SAE}=32 SAE at nearly the same FVU_w (Table 5); across architectures, FVU_w does not order downstream loss. Fibers trained on top of the frozen discrete code recover 99.9% while leaving the code exactly as it was, at the price of 640 continuous numbers per token.

6.8 Automated interpretability

Per-latent lenses mis-measure classifier codes, whose content lives in partitions rather than latent magnitudes. Blinded detection F1 of latent descriptions tracks code density, not architecture (correlation −0.89 with log density across six models), and the Ontologizer sits on the SAE trend. Much of that is language: a sparse code's latent whose top rows share one language is described by naming it and detected at F1 0.90–0.97 against random rows, and on features spanning as many languages as random rows do, the sparse SAE's lead over the Ontologizer shrinks from 0.17 to 0.10. Descriptions decoded from parameters alone fail outright (F1 0.04 for the Ontologizer and at most 0.14 for any model, against an always-match null of 0.67): a decoded atom is an instance of what an entry writes, not a criterion for it, and naming an entry requires activation evidence.

6.9 Against the desiderata

Scored against the introduction. *Causal*: positive with a qualification. Entries are realized by steps in embedding space and steer decodes, but an input-space step disturbs other heads exactly as much as a random step of its length, and the residual cascade doubles that. *Composable*: positive for interventions set on heads directly, which add. *Closed-form*: negative so far, since parameter-space decodes do not describe and heads do not own disjoint blocks of the dictionary that a static reading could take one at a time. Underneath all three sits identifiability, which the proposal took for granted: a head that is not the same head on the next run cannot be named, and past the first layer no head is. On reconstruction the verdict has turned: the soft code pays heavily for its structure, but the hard code beats `TopK` SAEs held to the same total bits on SONAR, though not an entropy-coded L1 SAE, and beats one in downstream loss on GPT-2.

7 Future work

7.1 Identifiability first

Before heads are named they must be reproducible, or it must be known which are. The first layer partly is, and one head in it recurs in every run; what lets that head recur when no other does, and whether training pressure can extend it, are the first questions.

7.2 Steering at scale

Supervised steering vectors are now scored beside each model's own directions on that model's categories (working notes) and, for one softmax model, on language (Section 6.5). Still open: the language comparison on the hard codes; bilinear eigenfeatures as steering directions; whether one entry's effect is consistent across contexts; and whether the flat model's near-perfect realization at half the stack's collateral is worth its 1.79× reconstruction cost. Collateral needs a lever more than a better direction: the hard code's heads sit almost on their decision boundaries, and knowing the partition buys little, since supervised vectors disturb the other heads at most 0.1 less often than a random step of their length. Wider margins look like that lever. In a five-layer stack of two-entry heads, a step a quarter of the input's norm long would flip, to first order, 13% of the other heads in its layer, against 30% for the 32-entry stack, and measured it disturbs 17–19% of them in every direction, against 66%. That arm was already as hard to disturb, by both measures, at the 32-entry stack's step count, and it matched that stack's mean-entropy pressure per head. But it trained on one seed and at half the batch size, so it does not yet isolate the number of entries. The single-layer hard variant `g160top1`, whose decode direction is the one that avoids the other heads (Section 6.4), is the reference to beat.

7.3 Task reconstruction

Recover what each reproducible head classifies: partition-against-label mutual information led by topic and genre, which are what the embedding is organized by; descriptions contrasting an entry with its siblings, validated by blinded assignment accuracy against $1/k$; and parallel-text invariance to separate content heads from form heads.

7.4 Disjoint heads

No trained model partitions its coordinates between heads, and none has trained with the support overlap itself as a loss term. Whether that penalty can buy the partition, and at what reconstruction price, is open; concatenated heads, disjoint by construction, are the comparison that already has it.

7.5 Transcoder adaptation

The architecture currently reconstructs its own input. Adapting it to predict a sublayer's output from its input, as transcoders do for MLPs [16], is the obvious next target. The residual stack then needs a forwarding mode that does not read the target, since the target is unavailable at inference.

Acknowledgements

Design, typesetting, and editorial production of this preprint are by The Flock Capital Research Department, whose error-diffusion pipeline also printed the plates.

References

[1] Artem Babenko and Victor Lempitsky. "Additive Quantization for Extreme Vector Compression". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2014, pp. 931–938. URL: https://openaccess.thecvf.com/content_cvpr_2014/html/Babenko_Additive_Quantization_for_2014_CVPR_paper.html.

[2] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. *Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation*. 2013. arXiv: 1308.3432 [cs.LG]. URL: https://arxiv.org/abs/1308.3432.

[3] Nick Bostrom. *Superintelligence: Paths, Dangers, Strategies*. Oxford: Oxford University Press, 2014. ISBN: 978-0-19-967811-2.

[4] Samuel R. Bowman et al. *Measuring Progress on Scalable Oversight for Large Language Models*. 2022. arXiv: 2211.03540 [cs.HC]. URL: https://arxiv.org/abs/2211.03540.

[5] Trenton Bricken et al. "Towards Monosemanticity: Decomposing Language Models With Dictionary Learning". In: *Transformer Circuits Thread* (2023). URL: https://transformer-circuits.pub/2023/monosemantic-features/index.html.

[6] Collin Burns et al. "Discovering Latent Knowledge in Language Models Without Supervision". In: *International Conference on Learning Representations*. 2023. URL: https://arxiv.org/abs/2212.03827.

[7] Bart Bussmann et al. "Learning Multi-Level Features with Matryoshka Sparse Autoencoders". In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025, pp. 6077–6101. arXiv: 2503.17547 [cs.LG]. URL: https://arxiv.org/abs/2503.17547.

[8] Tue M. Cao et al. *Tree SAE: Learning Hierarchical Feature Structures in Sparse Autoencoders*. 2026. arXiv: 2605.07922 [cs.LG]. URL: https://arxiv.org/abs/2605.07922.

[9] Lawrence Chan, Leon Lang, and Erik Jenner. *Natural Abstractions: Key Claims, Theorems, and Critiques*. 2023. URL: https://www.lesswrong.com/posts/gvzW46Z3BsaZsLc25/natural-abstractions-key-claims-theorems-and-critiques-1.

[10] David Chanin et al. "A is for Absorption: Studying Feature Splitting and Absorption in Sparse Autoencoders". In: *Advances in Neural Information Processing Systems*. Vol. 38. 2025, pp. 91489–91526. arXiv: 2409.14507 [cs.CL]. URL: https://arxiv.org/abs/2409.14507.

[11] Bilal Chughtai, Lawrence Chan, and Neel Nanda. "A Toy Model of Universality: Reverse Engineering How Networks Learn Group Operations". In: *Proceedings of the 40th International Conference on Machine Learning*. Vol. 202. Proceedings of Machine Learning Research. PMLR, 2023, pp. 6243–6267. URL: https://proceedings.mlr.press/v202/chughtai23a.html.

[12] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. 2nd ed. Wiley, 2006.

[13] James P. Crutchfield. "Way More than the Sum of Their Parts: From Statistical to Structural Mixtures". In: *Entropy* 28.1 (2026), p. 111. DOI: 10.3390/e28010111.

[14] Hoagy Cunningham et al. *Sparse Autoencoders Find Highly Interpretable Features in Language Models*. 2023. arXiv: 2309.08600 [cs.LG]. URL: https://arxiv.org/abs/2309.08600.

[15] Evan Duan. *Pre-Intervention Prediction of Sparse Autoencoder Steering Side Effects*. 2026. arXiv: 2606.08365 [cs.LG]. URL: https://arxiv.org/abs/2606.08365.

[16] Jacob Dunefsky, Philippe Chlenski, and Neel Nanda. "Transcoders Find Interpretable LLM Feature Circuits". In: *Advances in Neural Information Processing Systems*. Vol. 37. 2024. arXiv: 2406.11944. URL: https://arxiv.org/abs/2406.11944.

[17] Paul-Ambroise Duquenne, Holger Schwenk, and Benoît Sagot. *SONAR: Sentence-Level Multimodal and Language-Agnostic Representations*. 2023. arXiv: 2308.11466 [cs.CL]. URL: https://arxiv.org/abs/2308.11466.

[18] Leo Gao et al. *Scaling and evaluating sparse autoencoders*. 2024. arXiv: 2406.04093 [cs.LG]. URL: https://arxiv.org/abs/2406.04093.

[19] Ryan Greenblatt et al. *Alignment Faking in Large Language Models*. 2024. arXiv: 2412.14093 [cs.AI]. URL: https://arxiv.org/abs/2412.14093.

[20] Jesse Hoogland et al. "Loss Landscape Degeneracy and Stagewise Development in Transformers". In: *Transactions on Machine Learning Research* (2025). arXiv: 2402.02364 [cs.LG]. URL: https://openreview.net/forum?id=45qJyBG8Oj.

[21] Eric Jang, Shixiang Gu, and Ben Poole. "Categorical Reparameterization with Gumbel-Softmax". In: *International Conference on Learning Representations*. 2017. arXiv: 1611.01144. URL: https://arxiv.org/abs/1611.01144.

[22] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. "Product Quantization for Nearest Neighbor Search". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33.1 (2011), pp. 117–128.

[23] Anton Korznikov et al. *OrtSAE: Orthogonal Sparse Autoencoders Uncover Atomic Features*. 2025. arXiv: 2509.22033 [cs.LG]. URL: https://arxiv.org/abs/2509.22033.

[24] LCM team et al. *Large Concept Models: Language Modeling in a Sentence Representation Space*. 2024. arXiv: 2412.08821 [cs.CL]. URL: https://arxiv.org/abs/2412.08821.

[25] Chen-Yu Lee et al. "Deeply-Supervised Nets". In: *Proceedings of the Eighteenth International Conference on Artificial Intelligence and Statistics*. Vol. 38. Proceedings of Machine Learning Research. PMLR, 2015, pp. 562–570. URL: https://proceedings.mlr.press/v38/lee15a.html.

[26] Aleksandar Makelov, Georg Lange, and Neel Nanda. *Is This the Subspace You Are Looking for? An Interpretability Illusion for Subspace Activation Patching*. 2023. arXiv: 2311.17030 [cs.LG]. URL: https://arxiv.org/abs/2311.17030.

[27] Samuel Marks et al. *Auditing Language Models for Hidden Objectives*. 2025. arXiv: 2503.10965 [cs.AI]. URL: https://arxiv.org/abs/2503.10965.

[28] Anish Mudide et al. *Efficient Dictionary Learning with Switch Sparse Autoencoders*. 2025. arXiv: 2410.08201 [cs.LG]. URL: https://arxiv.org/abs/2410.08201.

[29] Neel Nanda et al. *A Pragmatic Vision for Interpretability*. 2025. URL: https://www.alignmentforum.org/posts/StENzDcD3kpfGJssR/a-pragmatic-vision-for-interpretability.

[30] Neel Nanda et al. "Progress measures for grokking via mechanistic interpretability". In: *International Conference on Learning Representations*. 2023. arXiv: 2301.05217 [cs.LG]. URL: https://arxiv.org/abs/2301.05217.

[31] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. "Neural Discrete Representation Learning". In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017. arXiv: 1711.00937. URL: https://arxiv.org/abs/1711.00937.

[32] Kiho Park, Yo Joong Choe, and Victor Veitch. "The Linear Representation Hypothesis and the Geometry of Large Language Models". In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 39643–39666. arXiv: 2311.03658 [cs.CL]. URL: https://arxiv.org/abs/2311.03658.

[33] Kiho Park et al. "The Geometry of Categorical and Hierarchical Concepts in Large Language Models". In: *International Conference on Learning Representations*. 2025. arXiv: 2406.01506 [cs.CL]. URL: https://arxiv.org/abs/2406.01506.

[34] Gonçalo Paulo et al. "Automatically Interpreting Millions of Features in Large Language Models". In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025. arXiv: 2410.13928 [cs.LG]. URL: https://arxiv.org/abs/2410.13928.

[35] Michael T. Pearce et al. "Bilinear MLPs enable weight-based mechanistic interpretability". In: *International Conference on Learning Representations*. 2025. arXiv: 2410.08417 [cs.LG]. URL: https://arxiv.org/abs/2410.08417.

[36] Alec Radford et al. *Language Models are Unsupervised Multitask Learners*. Tech. rep. OpenAI, 2019.

[37] Abhay Sheshadri et al. *Why Do Some Language Models Fake Alignment While Others Don't?* 2025. arXiv: 2506.18032 [cs.LG]. URL: https://arxiv.org/abs/2506.18032.

[38] Andrew Wagenmaker et al. *Steering Your Diffusion Policy with Latent Space Reinforcement Learning*. 2025. arXiv: 2506.15799 [cs.RO]. URL: https://arxiv.org/abs/2506.15799.

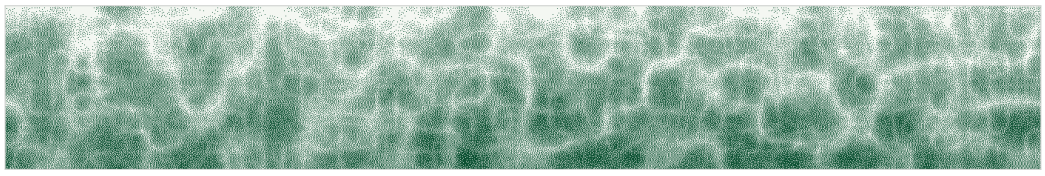
[39] Junxuan Wang et al. *Towards Universality: Studying Mechanistic Similarity Across Language Model Architectures*. 2024. arXiv: 2410.06672 [cs.CL]. URL: https://arxiv.org/abs/2410.06672.

[40] Zhengxuan Wu et al. "AxBench: Steering LLMs? Even Simple Baselines Outperform Sparse Autoencoders". In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025, pp. 67035–67080. arXiv: 2501.17148 [cs.CL]. URL: https://arxiv.org/abs/2501.17148.

[41] Zhen Xu et al. *Beyond Redundancy: Diverse and Specialized Multi-Expert Sparse Autoencoder*. 2025. arXiv: 2511.05745 [cs.LG]. URL: https://arxiv.org/abs/2511.05745.

[42] Neil Zeghidour et al. *SoundStream: An End-to-End Neural Audio Codec*. 2021. arXiv: 2107.03312 [cs.SD]. URL: https://arxiv.org/abs/2107.03312.

[43] Zhengyan Zhang et al. "MoEfication: Transformer Feed-forward Layers are Mixtures of Experts". In: *Findings of the Association for Computational Linguistics: ACL 2022*. Dublin, Ireland: Association for Computational Linguistics, 2022, pp. 877–890. DOI: 10.18653/v1/2022.findings-acl.71. URL: https://aclanthology.org/2022.findings-acl.71/.



RIDGE · SEED 32 · THE SAME PIPELINE, CLOSING THE FILE